Great — glad you're liking the direction!

Since Alley Pack is a development/automation tool, making it as self-contained and easy to distribute as possible makes total sense. Let's adjust the architecture to match what you described:

- AlleyPack.exe is a standalone .NET console app (targeting .NET Framework 4.8 or .NET 6+ for easier distribution if you prefer).
- It references **WixSharp** via NuGet (specifically the WiX v4 compatible version, which is the actively maintained path in 2026).
- It does **not** embed or ship the WiX binaries/compilers itself.
- Instead, when you run `AlleyPack.exe build` (or similar), it:
  1. Parses `alley-pack.yaml`
  2. Builds the WixSharp `Project` object in memory (exactly as we've been designing)
  3. Uses `Compiler.BuildMsi(project)` (or `Compiler.BuildBundle(...)` later) — this internally orchestrates the WiX toolchain
  4. But crucially: **the actual WiX tools (candle.exe, light.exe, wix.exe, etc.) must be available on the machine** where AlleyPack runs.

How to Make WiX Tools Available (Recommended Setup)

WixSharp (especially the modern `WixSharp_wix4` package) expects the WiX v4 toolchain to be installed globally or discoverable. The cleanest ways for a dev tool like AlleyPack are:

Option A – Strongly Recommended for Dev Machines (cleanest & most reliable)

- Require users (you, testers, devs) to install WiX v4 via dotnet tool:

  ```text
  dotnet tool install --global wix
  ```

  (This is the official way now; it puts `wix.exe` on PATH.)

- AlleyPack can even check for it at startup and give a nice error message:

```text
WiX toolchain not found. Please run: dotnet tool install --global wix
```

Option B – If You Want Zero Manual Installs (more work, but possible)

- Include the legacy `WixSharp.wix.bin` NuGet in your AlleyPack project (it bundles older WiX v3 binaries).
- Or manually bundle the WiX v4 binaries (extract from the WiX release zip) into your AlleyPack output folder and set `Compiler.WixLocation = Path.Combine(AppContext.BaseDirectory, "wix-bin");`
- This makes AlleyPack truly portable/single-file-ish, but:
  - WiX v4 licensing changed (some parts commercial for redistribution in certain cases — check current terms).
  - Bundling increases exe size significantly (~10-20 MB extra).
  - Harder to stay up-to-date with WiX patches.

Most real-world WixSharp-based tools (samples, CI scripts, etc.) go with **Option A** — assume WiX is installed globally via `dotnet tool`. It's what the WixSharp author recommends in recent docs/wiki.

Updated Project Setup for AlleyPack

Create a new console project:

```bash
dotnet new console -n AlleyPack -f net48   # or net6.0 / net8.0 if you want modern runtime
cd AlleyPack
```

Add packages (use latest stable as of March 2026):

```bash
```

```
dotnet add package WixSharp_wix4          # ~2.12.x or newer
dotnet add package YamlDotNet             # for YAML parsing
# Optional extras later: CommandLineParser or System.CommandLine for CLI
args
```

Your `AlleyPack.csproj` will look something like:

```XML
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net48</TargetFramework> <!-- or net8.0 -->
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="WixSharp_wix4" Version="2.12.2" /> <!--
bump to latest -->
    <PackageReference Include="YamlDotNet" Version="*" />
  </ItemGroup>
</Project>
```

High-Level Flow in `Program.cs`

```C#
using System;
using System.IO;
using WixSharp;
using YamlDotNet.Serialization;
using YamlDotNet.Serialization.NamingConventions;

class Program
{
    static int Main(string[] args)
    {
        if (args.Length == 0 || args[0] == "help")
        {
            Console.WriteLine("AlleyPack v0.1");
            Console.WriteLine("Usage: AlleyPack build [path-to-yaml]");
            Console.WriteLine("       AlleyPack --new-guid");
```

```
            Console.WriteLine("AlleyPack - new guid");
            return 0;
        }

        try
        {
            string yamlPath = args.Length > 1 ? args[1] : "alley-
pack.yaml";
            if (!File.Exists(yamlPath))
            {
                Console.Error.WriteLine($"YAML file not found:
{yamlPath}");
                return 1;
            }

            var config = LoadConfig(yamlPath);
            var project = BuildWixProject(config,
Path.GetDirectoryName(yamlPath)!);

            // Optional: project.OutFileName = $"Alley-
{config.Product.Name}-{config.Product.Version}.msi";

            Console.WriteLine("Building MSI...");
            string msiPath = Compiler.BuildMsi(project);
            Console.WriteLine($"Success! MSI created: {msiPath}");

            return 0;
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Error: {ex.Message}");
            return 1;
        }
    }

    static AlleyPackConfig LoadConfig(string yamlPath)
    {
        var deserializer = new DeserializerBuilder()
            .WithNamingConvention(CamelCaseNamingConvention.Instance)
            .Build();

        var yaml = File.ReadAllText(yamlPath);
        return deserializer.Deserialize<AlleyPackConfig>(yaml);
    }
```

```
    }

    static Project BuildWixProject(AlleyPackConfig config, string baseDir)
    {
        // This is where we map YAML → WixSharp Project
        // We'll fill this in detail next (aliases, tree building, etc.)
        var project = new Project(config.Product.Name)
        {
            Manufacturer = config.Product.Manufacturer,
            Version = new Version(config.Product.Version),
            UpgradeCode = new Guid(config.Product.UpgradeCode),
            // GUID = ... auto or from config
            Scope = config.Install.Scope == "perUser" ?
InstallScope.perUser : InstallScope.perMachine,
            // etc.
        };

        // TODO: resolve aliases, build Dir tree recursively from
config.Structure

        return project;
    }
}
```

Next Steps – Let's Fill the Gaps

1. Define the C# model classes (`AlleyPackConfig` , `ProductSection` , `InstallSection` , `SourceAliases` , `FolderEntry` , etc.) that match our finalized YAML.
2. Implement the alias resolver + Git root detection logic.
3. Write the recursive `Dir` builder that turns `structure` into nested `Dir` / `Files` objects.

Reply with:

- Which target framework you prefer (`net48` to match your project, or `net8.0` for single-file publish ease)?
- Whether you want to start with Option A (assume `wix` dotnet tool installed) or try Option B (bundled WiX)?

- If you're ready, say "go ahead with the model classes + alias resolver code"

We'll get you a compilable starter in the next round!